Biometric ID: Fuzzy Extractor

Dalia Papuc - dalia.papuc@epfl.ch
Project coordinator: Serge Vaudenay - serge.vaudenay@epfl.ch

February 2, 2022

1 Introduction

User authentication using biometric features, such as iris, fingerprint, finger-vein, is a topic of importance in today's time as we, as users, are exposed to it on a daily basis. Consider for instance unlocking your phone/tablet/laptop multiple times a day by means of your fingerprint or face. In this report, we focus on the finger-vein feature for authentication which uses the vein pattern of the finger, underneath the skin. This, in contrast to fingerprints, for instance, is not left behind when we touch a surface.

We make use of a Fuzzy Extractor to derive a key from finger-vein biometry. Fuzzy Extractors convert noisy non-uniform inputs into reliably reproducible, uniform random strings. A Fuzzy Extractor consists of two algorithms, *Generate* (*Gen*) and *Reproduce* (*Rep*). *Gen* takes as input a reading of the finger-vein biometric feature (the model) in our case and it outputs a uniform random string and a public helper string. While *Rep* makes use of the helper string and another reading of the biometric feature (the probe). In case the probe is sufficiently similar to the model, it outputs the same extracted string. We use *Gen* hence at enrolment, when we enrol a user with their biometric data, and *Rep* to authenticate the user whenever desired.

The noise tolerance of a Fuzzy Extractor is a property of importance when considering biometric data as we can imagine there are variations in data whenever we perform another reading. Another aspect of uttermost importance when dealing with such sensitive data is the notion of reusability since biometric data cannot be changed. We would like to preserve the security of the extracted string even though the same biometric feature is used to enrol with multiple (independent) organisations, i.e., given access to all public helper values and the extracted strings of other enrolments. Consequently, the report approaches two problems: (a) minimising the noise of data and (b) finding a Fuzzy Extractor scheme that provides reusability for finger-vein biometry.

A Fuzzy Extractor is able to tolerate errors yet we would like to allocate these errors to the variation in the finger-vein pattern rather than a variation in the finger pose. To this end, we propose and implement four different alignment functions which aim to align the model and a probe yet without having access

to the model. Having access to the model would make the task at hand much easier as we can identify and use reference points from the model and shift the probe according to these. We also present an evaluation that indicates the method that performs best.

For the Fuzzy Extractor scheme, we perform a literature study to identify our requirements and determine which schemes are suitable candidates for our needs. We wish a scheme that we can apply to finger-vein patterns, that is hard to invert and reusable. We opt for the construction in [1]. The construction makes use of digital lockers, computationally secure symmetric encryption schemes. It requires a source with high-entropy samples, i.e., the sub-string formed by the bits at k randomly chosen positions in input data should have enough min-entropy. To fulfil this criterion, we use as a source the biometric hash of the finger-vein patterns obtained after the feature extraction. We estimate the parameter value for the scheme given our dataset and find out the security is quite low, i.e., 40 bits, and requires a high number of iterations, i.e., 3.2×10^9. We propose a few approaches to improve these results.

We would like to emphasise that the numerical results discussed in this report should be taken with a grain of salt given the size of the dataset, i.e., 2 users. Yet the methods followed can definitely be reused once a larger dataset is obtained.

We proceed by discussing Fuzzy Extractors in Section 2, namely, relevant notions to the rest of the report and results from the literature. We continue with the description of the chosen Reusable Fuzzy Extractor construction in Section 3. In Section 4 we discuss the four proposals for alignment after which we conclude.

2 Fuzzy Extractors

This section starts by introducing concepts referred to throughout the report (subsection 2.1). It continues by discussing relevant results from literature (subsection 2.2) which leads us to the discussion of reusability and our choice of Fuzzy Extractor construction (subsection 2.3).

2.1 Preliminaries and definitions

General

A *metric space* is a set $\mathcal{M}$ with a distance function $dis : \mathcal{M} \times \mathcal{M} \to \mathbb{R}^+$ e.g., Hamming metric, set difference metric, edit metric. In this work we consider the *Hamming metric*: $\mathcal{M} = \mathcal{F}^n$, for some alphabet $\mathcal{F}$, and $dis(w, w')$ is the number of positions in which the strings w and w' differ.

The *ball of radius* t around x for a metric space $(\mathcal{M}, dis)$ is the set of all points within radius t:

$$B_t(x) = \{y | dis(x, y) \leq t\}$$

The *statistical distance* between random variables X and Y with the same domain is defined as:

$$\mathbf{SD}(X,Y) = \frac{1}{2}\sum_{x} |\Pr(X=x) - \Pr(Y=x)|$$

The *computational distance* between two random variables X and Y for a distinguisher D (or a class of distinguishers $\mathcal{D}$):

$$\delta^D(X,Y) = |\mathbb{E}[D(X)] - \mathbb{E}[D(Y)]|$$

.

The best strategy for an adversary to guess a secret, e.g., a random variable, is to guess the most likely value. Hence, the *predictability* of random variable X is $\max_x \Pr[X=x]$ and accordingly the *min-entropy* is:

$$\mathbf{H}_\infty(X) = -\log(\max_x \Pr[X=x])$$

According to [2], the min-entropy of a distribution indicates how many nearly uniform random bits can be extracted from it.

Consider the pair of random variables X, Y. If an adversary discovers the value y of Y, then predictability of X becomes $\max_x \Pr[X=x|Y=y]$. On average, the adversary's chance of success in predicting X is then $\mathbb{E}_{y\in Y} \max_x \Pr[X = x|Y=y])$ [2]. The *average (conditional) min-entropy* of X given Y is:

$$\tilde{\mathbf{H}}_\infty(X|Y) = -\log(\mathbb{E}_{y\in Y} \max_x \Pr[X=x|Y=y])$$

Secure Sketches and Fuzzy Extractors

Secure Sketches allow to reconstruct a noisy input: given a sketch s and an element close to the input, we can recover the input.

Definition 1. *(Secure Sketch [3]) An $(\mathcal{M}, m, \tilde{m}, t)$-secure sketch with error δ is a pair of randomised procedures, sketch (SS) and recover (Rec), with the following properties:*

Sketch (SS): takes as input $w \in \mathcal{M}$ and outputs a bit string $s \in \{0,1\}^*$

Recover (Rec): takes as input an element $w' \in \mathcal{M}$ and a bit string $s \in \{0,1\}^*$

Correctness: if $dis(w,w') \leq t$, then $\Pr[Rec(w', SS(w)) = w] \geq 1-\delta$ (probability over coins SS and Rec)

Security: for any distribution W over $\mathcal{M}$ with min-entropy m, the value of W can be recovered by the adversary who observes $SS(W)$ with probability no greater than $2^{-\tilde{m}}$. That is, $\tilde{\mathbf{H}}_\infty(W|SS(W)) \geq \tilde{m}$.

A Secure Sketch is efficient if *SS* and *Rec* run in expected polynomial-time.

Fuzzy Extractors covert noisy non-uniform inputs into reliably reproducible, uniform random strings. We present three different security notions for Fuzzy Extractors, Definitions 2–4. We define the information-theoretic variant of a Fuzzy Extractor in Definition 2 and the computational variant of a Fuzzy Extractor in Definition 3. Using the computational variant, we construct the definition of a Reusable Fuzzy Extractor (Definition 4).

Definition 2. *(Fuzzy Extractor [3]) An $(\mathcal{M}, m, \ell, t, \epsilon)$-fuzzy extractor with error δ is a pair of randomised procedures, generate (Gen) and reproduce (Rep), with the following properties:*

Generate (Gen): takes as input $w \in \mathcal{M}$, and outputs an extracted string $r \in \{0,1\}^\ell$, and a (public) helper string $p \in \{0,1\}^*$

Reproduce (Rep): takes as input an element $w' \in \mathcal{M}$ and a bit string $p \in \{0,1\}^*$

Correctness: $\forall (w, w')$ s.t. $dis(w, w') \leq t$, for $Gen(w) = (R, P)$, then $Rep(w', P) = R$ with probability (over the coins of Gen, Rep) at least $1 - \delta$. If $dis(w, w') > t$, then no guarantee is provided about the output of Rep

Security: $\forall$ distribution W on $\mathcal{M}$ of min-entropy m, the string R is nearly uniform even for those who observe P: if $Gen(w) = (R, P)$, then

$$\mathbf{SD}((R, P), (U_\ell, P)) \leq \epsilon$$

where U_ℓ: uniform distribution on $\ell-$bit binary strings and $\mathbf{SD}(X, Y)$ is the statistical distance between X, Y.

A Fuzzy Extractor is efficient if *Gen* and *Rep* run in expected polynomial-time.

Definition 3. *(Computational Fuzzy Extractor [3]) Let $\mathcal{W}$ be a family of probability distributions over $\mathcal{M}$. An $(\mathcal{M}, \mathcal{W}, \ell, t)$-computational fuzzy extractor is (ϵ, s_{sec}) with error δ if the pair of randomised procedures, generate (Gen) and reproduce (Rep) are a fuzzy-extractor with the following security property instead:*

Security: $\forall W \in \mathcal{W}$, the string R is pseudorandom conditioned on P:

$$\delta^{\mathcal{D}_{s_{sec}}}((R, P), (U_\ell, P)) \leq \epsilon$$

where $\delta^{\mathcal{D}}(X, Y)$ is the computational distance between X, Y for a distinguisher $\mathcal{D}$ and $\mathcal{D}_{s_{sec}}$ the class of randomised circuits which output a single bit and have size at most s_{sec}.

Definition 4. *(Reusable Fuzzy Extractor [1]) Let $\mathcal{W}$ be a family of distributions over $\mathcal{M}$. Let (Gen, Rep) be a $(\mathcal{M}, W, \ell, t)-$computational fuzzy extractor that is $(\epsilon, s_{sec})-$hard with error δ. Let $(W^1, W^2, ..., W^\rho)$ be ρ correlated random variables such that each $W^j \in \mathcal{W}$. Let D be an adversary. Define the following game for all $j = 1, \ldots, \rho$:*

Sampling: The challenger samples $w^j \leftarrow W^j$ and $u \leftarrow \{0,1\}^\ell$

Generation: The challenger computes $(r^j, p^j) \leftarrow Gen(w^j)$

Distinguishing: The advantage of D is:

$$\begin{aligned} Adv(D) = &\Pr[D(r^1, ..., r^{j-1}, r^j, r^{j+1}, ..., r^\rho, p^1, ..., p^\rho) = 1] \\ &- \Pr[D(r^1, ..., r^{j-1}, u, r^{j+1}, ..., r^\rho, p^1, ..., p^\rho) = 1] \end{aligned}$$

(Gen, Rep) is $(\rho, \epsilon, s_{sec})-$reusable if for all $D \in D_{sec}$ and all $j = 1, \ldots, \rho$, the advantage is at most ϵ.

Digital Lockers

We introduce the definition of a Digital Locker. A Digital Locker consists of two algorithms *lock* and *unlock*. The *lock* algorithm locks value *val* using a key *key* and it returns c. The *unlock* algorithm attempts to unlock c by using a key, key'. When $key = key'$ it returns *val* with a high probability, otherwise it returns $\perp$ with a high probability. Security is defined following the Virtual Grey Box property [4]. The simulator S may perform $q(\lambda)$ queries to an oracle, i.e., ideal locker. The security property indicates that if the keys to the ideal locker are hard to guess, S cannot unlock and neither can the adversary, $\mathcal{A}$. The oracle, $idealUnlock(key, val)$, returns *val* when given *key*, and $\perp$ otherwise [1, 5]. Let f be polynomially bounded. Then:

Definition 5. *(f-Composable Secure Digital Locker [1]) The pair of algorithm $(lock, unlock)$ with security parameter λ is an $f-$composable secure digital locker with error γ if the following hold:*

Correctness: For all key and val, $\Pr[unlock(key, lock(key, val)) = val] \geq 1 - \gamma$

Wrong key detection: For any $key' \neq key$ and any val, $\Pr[unlock(key', lock(key, val)) = \perp] \geq 1 - \gamma$

Security: For every PPT adversary $\mathcal{A}$ and every positive polynomial p, there exists a (possibly inefficient) simulator S and a polynomial $q(\lambda)$ such that for any sufficiently large s, any sequence of values (val_i, key_i) for $i = 1, \ldots, f$, and any auxiliary input $z \in \{0,1\}^*$,

$$\begin{aligned} &|\Pr[\mathcal{A}(z, \{lock(key_i, val_i)\}_{i=1}^f) = 1] \\ &- \Pr[S^{\{idealUnlock(key_i, val_i)\}_{i=1}^f}(z, \{|key_i|, |val_i|\}_{i=1}^f) = 1]| \leq \frac{1}{p(s)} \end{aligned}$$

where S is allowed $q(\lambda)$ oracle queries to the oracles $\{idealUnlock(key_i, val_i)\}_{i=1}^f$.

2.2 Literature results

The security property of the information-theoretic Fuzzy Extractor, Definition 2, requires the extracted string r to be uniform to anyone who observes the public string p; this includes a computationally unbounded adversary. The authors of [6] observed that this property is harder to satisfy as t increases. This is because it becomes easier for an adversary to guess r by guessing a w' within distance t of w and run *Rep* (w', p) [6]. In [6], the authors define the notion of *fuzzy min-entropy* which measures the hardness of executing this attack.

Fuzzy min-entropy indicates the probability of guessing a value within distance t of the original reading w, sampled from a distribution W [6]:

$$H_{t,\infty}^{fuzz}(W) = -\log \max_{w'} \Pr[W \in B_t(w')]$$

with $B_t(w')$ the ball of radius t around w'.

The authors further prove that whenever the probability distribution function W is known to a Fuzzy Extractor designer, fuzzy min-entropy is a necessary and sufficient condition for constructing a Fuzzy Extractor. They prove that the extracted string r is about the size of $H_{t,\infty}^{fuzz}(W)$.

However, it is not always feasible to assume a designer has precise knowledge of W. High-entropy distributions need to be modelled as they can not be completely observed [6]. We have to consider that the adversary's model may be better than the designer's model (Distributional Uncertainty setting). In this setting, the authors of [6] prove that one cannot design a Fuzzy Extractor that works for all possible sources. If we only know that distribution comes from $\mathcal{W}$, we cannot fully secure the family, despite the existence of a separate design for each distribution in the family [6].

We define:

$$\mu \text{ (entropy rate) } = H_{t,\infty}^{fuzz}(W)/n$$

and summarise the main (negative) results of [6]:

- *Rep* (perfectly correct) - rules out Fuzzy Extractors for entropy rates as high as $\mu \approx 0.18$
- *Rep* (allowed to make mistakes) - rules out Fuzzy Extractors for entropy rates as high as $\mu \approx 0.07$
- *Rec* (allowed to make mistakes) - rules out Secure Sketches for entropy rates as high as $\mu \approx 0.5$

These results increase the lower bound for the amount of fuzzy min-entropy needed in a physical distribution in order to only base the security on fuzzy min-entropy. To avoid the aforementioned negative results, we can follow one of the approaches:

1. Focus on providing security for high entropy distributions only
2. Restrict adversary: assume it runs in polynomial time
   (a) computational Secure Sketch $\implies$ IT Secure Sketch [3]
   (b) decoding random codes, e.g., [3]
   (c) security of hash function, e.g., [1, 5]
   (d) hardness of discrete log, e.g., [7, 8]
   (e) general cryptographic primitives, e.g., [9]
3. Assume additional structure from the distribution
   (a) random subsets of dimensions have high entropy, e.g., [1]
   (b) independence between dimensions, e.g., [10]

Let us discuss these results in conjunction with human biometrics and in particular finger-vein biometrics. For item 1 to hold, we would need to model the finger-vein distribution and check whether it is indeed a high entropy distribution. Given we do not currently have access to a large number of samples, we cannot model nor estimate this distribution accurately.

Furthermore, we cannot rely on a universal entropy estimation. A first study aiming to estimate the entropy of finger-vein patterns is done in [11]. The authors consider two public databases and use three types of feature extractors. They conclude that the entropy of finger-vein patterns depends on the database and the feature extractor. Hence, we cannot use their results and would need to estimate finger-vein entropy using our own database and feature extractor.

We move to item 2 and discuss the first sub-point, i.e., 2a. A common approach in Fuzzy Extractor constructions is the *Sketch-then-Extract* approach. This means applying a Secure Sketch (Definition 1) and then a Randomness Extractor. Using the Secure Sketch, we can recover the original w from any nearby element w' ($dis(w, w') \leq t$), while using the Randomness Extractor on w, we can produce uniform bits.

In [3], the authors prove that computational Secure Sketches are subject to similar bounds as information-theoretic ones. The lower bound applies to Fuzzy Extractors as well whenever an algorithm can efficiently recover w from the output of *Rep*. However, Fuzzy Extractors do not need to recover w, they can simply output the same extracted string r. We should then consider constructions that are hard to invert [3, 12]. We will discuss the remaining sub-items of 2 when considering Reusable Fuzzy Extractor constructions.

The last sub-item, 3b, asks the sources to be independent and identically distributed (i.i.d.). However, many biometrics sources are not i.i.d., e.g., iris biometrics [13].

We conclude this subsection by considering adversaries constrained to run in polynomial time, we use Definition 3 to define a computational Fuzzy Extractor and we consider direct constructions, hard to invert, which includes not following the *Sketch-then-Extract* approach. We next dive into an important property to be satisfied by a Fuzzy Extractor construction for our use case: reusability.

2.3 Reusability for Fuzzy Extractors

Boyen [14] introduced the notion of reusability. He showed prior Fuzzy Extractor constructions to not be secure, i.e., they leak information on input data w when *Gen* is called multiple times on different readings of the same biometric data. Since we wish to allow users to securely repeat the enrolment procedure using the same finger(-vein), we consider constructions that fulfil reusability.

In detail, *Gen* may be called ρ times on correlated readings $w^1 \dots w^\rho$ of the same source, producing $(r^1, p^1), \dots, (r^\rho, p^\rho)$. Security should hold for each extracted string r^i even when all helper strings $p^1 \dots p^\rho$ are known. A even stronger security property requires the security for each string r^i to hold in the presence of not only $p^1 \dots p^\rho$, but also r^j, $j \neq i$ [1].

There are multiple definitions of Reusable Fuzzy Extractor in the literature. We refer the reader to Table 1 of paper [1] for a comparison between the different definitions.

We summarise the Reusable Fuzzy Extractor constructions found in the literature by the assumption the construction relies on:

1. decoding random codes: [15] ($\triangle$), [16]
2. security of hash function: [1, 5], [14] ($*$)
3. hardness of discrete log: [7, 8] ($*$)
4. general cryptographic primitives: [9] ($*$)

From these constructions and the conclusions of subsection 2.2, we decide to remove constructions based on Secure Sketch. We indicate these constructions by the symbol ($*$). Next, we remove all constructions for which it is unclear how they apply to human biometrics. We indicate these constructions by the symbol ($\triangle$). In particular, article [15] targets digital fingerprints. The authors wish to test these on human biometrics as part of future work yet we could not find a follow-up paper.

To choose among the remaining constructions, we would like to point out one criterion present in Reusable Fuzzy Extractor definitions: *correlation*. The Reusable Fuzzy Extractor definition in [1] makes no assumptions on how readings are correlated, i.e., the distributions $W_1 \dots W_\rho$ are arbitrarily correlated. While most other definitions (including [16]) assume multiple readings are correlated by a shift. By examining multiple captures of the same finger-vein biometric source we notice that multiple readings of the same source exhibit arbitrary correlations.

Finally, we are left with the constructions in [1, 5]. In article [5], the authors present a general framework where one uses a randomisation step, reusable pseudoentropic isometry (RPI), which provides reusability, and then one can apply a non-reusable Fuzzy Extractor. Article [1] presents three direct constructions, out of which Construction 1 is reusable. Both constructions make use of Digital Lockers.

Digital Lockers are computationally secure symmetric encryption schemes. They preserve security even when used multiple times with correlated and weak (non-uniform) keys [1, 5]. Learning anything about the plaintext is as hard as finding the key [1]. In the spirit of [1, 5], we denote $c = \mathit{lock}(\mathit{key}, \mathit{val})$ to indicate locking value val using the key key, $\mathit{unlock}(\mathit{key}, c)$ to indicate unlocking, i.e., obtaining value val when key is used and otherwise $\perp$ with high probability.

In [5], the authors provide a Reusable Fuzzy Extractor construction for the set difference metric. Which does not apply to our use case as we make use of the Hamming metric. Furthermore, they noticed that Construction 2 of [1] is an instantiation of their framework in the Hamming metric for specific sources. In particular, they require each symbol w_i (consider $w = w_1 \dots w_s$) to individually have entropy (where the original construction does not assume all w_i have entropy) [5]. Construction 1 of [1] requires sources with high-entropy samples.

That is, they require the sub-string formed by the bits at k randomly chosen positions in w to have enough min-entropy. Note that this corresponds to the assumption in item 3a, i.e., random subsets of dimensions have high entropy.

One concern for both of these assumptions is whether our finger-vein source actually fulfils the assumptions for the source. We opt for Construction 1 of [1] and proceed by describing it in more detail in the upcoming section. We also discuss solutions for the aforementioned concern.

3 The Construction

In this section, we first discuss the requirement on the source, and then we present the construction. We refer the reader to subsection 2.1 for the formal definitions of a Reusable Fuzzy Extractor (Definition 4) and Digital Locker (Definition 5), or to subsection 2.3 for an informal description.

3.1 Assumption on the source

Construction 1 of [1] requires sources with high-entropy samples.

Definition 6. *(Sources with High-Entropy Samples [1]) Let the source $W = W_1, \ldots, W_n$ consist of strings of length n over some arbitrary alphabet Z (particular case: binary alphabet $Z = \{0, 1\}$). For some parameters k, α we say that the source W is a source with $\alpha-$entropy $k-$samples if:*

$$\tilde{\mathbf{H}}_\infty(W_{j_1} \ldots W_{j_k} | j_1 \ldots j_k) \geq \alpha \text{ for uniformly random } 1 \leq j_1, \ldots, j_k \leq n.$$

Sampling preserves entropy rate. This means that a source with sufficient min-entropy, also has high-entropy samples [1]. Additionally, low-entropy sources may exhibit high-entropy samples (relative to the length of the string; the entropy of a sub-string may not exceed the entropy of the string otherwise) [1]. The authors give the example of a binary string obtained via signal processing from an underlying reading, e.g., an image of the iris, where the signal has a lot of redundancy, e.g., the nearby pixels are highly correlated [1].

We notice that our current feature extraction method yields an image with considerably more 0s than 1s. This raises the concern of whether the random sub-string obtained at k random positions will actually have enough entropy.

There are multiple ways to approach this problem. First, we can consider a region of interest, for example, the mask region (which represents the finger region) to reduce the number of 0 values. This method, however, does not fully resolve our issue; for example, a mask indicates a region of about 40000 elements, the entire image contains 90240 elements and the number of 1s is about 4000. Second, we can modify the feature extraction method, e.g., either its parameters or the method itself, so that we obtain a more uniform result after feature extraction. The first two methods can also be combined. Third, we can use the biometric hashed value of the extracted features instead of using the obtained features directly. This allows adjusting the data by requiring an

equal ratio of each value in the output of hashing. A final proposal is to remove some of the locations selected at random until we reach an equal number of 0s and 1s.

In our practical implementation, we make use of biometric hashing to approach this problem. An evaluation of all the aforementioned methods as well as considering new methods is a direction for future work.

3.2 Sample-then-Lock

We start by explaining how the construction works and then present the pseudocode in Construction 1.

The *Gen* procedure functions as follow. First, we pick a random string r which in combination with the helper string p defines the output of the procedure. We sample k random indices and take the symbol of the input data w (the model) at each location. We denote this sub-string by v. Using the Digital Locker, we lock the value r using key v. We denote by c the result of the *lock* operation and let p be the value c and the k indices previously selected. We repeat this process f times, each time locking the same value r. Finally, we output (r, p) where p aggregates the p values in all f runs.

The *Rep* procedure takes as input the helper string p and input data w' (the probe). For each iteration (f total), we first decompose p in the result of the *lock* procedure, c, and the k indices. Given these indices, we form the sub-string of w', call it v', and attempt to *unlock* c using v'. If the *unlock* procedure outputs a non-empty value, it means that we managed to reproduce value r.

Construction 1. (Sample-then-Lock) [1] Let Z be an alphabet, and let $W = W_1, ..., W_n$ be a source with $\alpha-$entropy $k-$samples, where each W_j is over Z. Let f be a parameter, to be determined later. Let (*lock*, *unlock*) be an $f-$composable secure digital locker with error γ (for $\ell-$bit keys and values over Z^k). Define *Gen*, *Rep* as:

Gen
Input: $w = w_1, \ldots, w_n$
Sample $r \xleftarrow{\$} \{0,1\}^\ell$
For $i = 1, \ldots, f$:
  Choose random $1 \leq j_{i,1}, \ldots, j_{i,k} \leq n$
  Set $v_i = w_{j_{i,1}}, \ldots, w_{j_{i,k}}$
  Set $c_i = \mathit{lock}(v_i, r)$
  Set $p_i = c_i, (j_{i,1}, \ldots, j_{i,k})$
Output: (r, p), where $p = p_1 \ldots p_f$.

Rep
Input: $(w' = w'_1, \ldots, w'_n, p = p_1 \ldots p_f)$
For $i = 1, \ldots, f$:
  Parse $p_i = c_i, (j_{i,1}, \ldots, j_{i,k})$
  Set $v'_i = w'_{j_{i,1}}, \ldots, w'_{j_{i,k}}$
  Set $r_i = \mathit{unlock}(v'_i, c_i)$
  If $r_i \neq \perp$:
    Output: r_i
Output: $\perp$.

In [17], the authors modify Construction 1 for implementation and prove it correct. For sampling key r, they use a random 256-bit string. The locking algorithm outputs a pair $\mathit{nonce}, HMAC(\mathit{nonce}, v) \oplus (0^{256}||r)$. The unlocking procedure first recomputes the hash and checks whether the XOR of the hash with the second element of the pair has a prefix of 0^{256}. When this is the case, it outputs the suffix, r. The implementation in both *C* and *Python* can be found at `https://github.com/benjaminfuller/CompFE`.

Parameter choice. The authors of [17] provide the following relation for estimating the parameters, the number of Digital Lockers f and the number of indices k to be randomly chosen which defines the size of the sub-string:

$$1 - \Big(1 - (1 - mean\ error\ rate)^k\Big)^f = correctness\ target$$

where the *correctness target* is for instance the desired True Positive Rate (TPR).

3.3 Parameter estimation

To be able to estimate the number of Digital Lockers f as well as the size of the sample k, we need to first estimate the mean error rate of the data used as input to the Fuzzy Extractor. In our case, the input data is the biometric hash of the finger-vein patterns.

To estimate the mean error rate we look at the intraclass comparisons, comparisons of the Hamming distance between two hashes of the same finger. The Hamming distance indicates the proportion of disagreeing components between two hashes. We use the index, middle and ring fingers of both the left and the right hand coming from the same person. Moreover, the device has two cameras and hence outputs two images to be processed. The dataset consists of 2 users (out of which 2 fingers for one user are mysteriously missing) and 2 attempts for each user, i.e., one attempt serves as the model while the other as a probe. This yields a total of 20 comparisons (see Appendix for the list of pairs).

Before we compute the Hamming distance between the model and a probe, we wish the probe to be as well-aligned to the model as possible to reduce the noise. For this, we compute the optimal shifting factor of the probe by calling the function *miurascore*. The function computes the best similarity score between the model and a probe while shifting the probe in (x, y) direction. It also outputs the shifting factors for which the match between the model and probe is best. Using these coordinates, we shift the probe.

We then compute the biometric hash of both the model and the (now shifted) probe using the (optimal) value for $d = 3500$. We keep ϵ set to 0 so that the number of 0s in the obtained hash is small. We obtain an array of mostly 1s and -1s and of similar proportion.

To compute the Hamming distance between two hashes we do the following. For each pair of hashes coming from the same finger, we compute the product of the hashes element-wise. When the values are equal in both hash the product is 1, i.e., $-1 \times -1 = 1 \times 1 = 1$. We compute the total number of 1s in the product which indicates the similarity between the hashes. We then subtract from the total length of hash, i.e., d, the number of 1s–this gives the number of different components. Finally, we divide this value by the length of the hash. We call the obtained value the Hamming distance. Alternatively, one could use the *scipy.spatial.distance.hamming* function in *Python*.

We compute the mean over all such values (Table 1) and obtain a mean error rate of 0.418643. We further use this value to estimate parameters for the

0.4105714285714286	0.42714285714285716	0.432
0.41514285714285715	0.46885714285714286	0.42742857142857144
0.42542857142857143	0.41914285714285715	0.44085714285714284
0.3742857142857143	0.4377142857142857	0.362
0.416	0.41114285714285714	0.42228571428571426
0.4062857142857143	0.3882857142857143	0.4357142857142857
0.42714285714285716	0.42542857142857143	

Table 1: The mean error rate: 0.41864285714285715

construction. We set a TPR of 70% and the equation becomes:

$$1 - \left(1 - (1 - 0.418643)^k\right)^f = 0.7$$

We experimentally set the sample size to $k = 40$, in this case, we need approximately $f = 3.2 \times 10^9$ lockers to obtain a target correctness of 70%. We note that f is rather large while the security is rather low. The values obtained are in Table 2.

Approaching the problem from another angle, we set the number of lockers to a smaller value, e.g., 10^8, we search for the maximum value of k and obtain $k = 33$ and a probability of 0.81 (Table 3). While we obtain a lower value for f, the security decreases even more.

Of course, we cannot take these numbers as representative given the small dataset. Yet let us perform an imagination exercise and consider these results as valid. If we wish to obtain higher security we can either:

(a) increase the number of lockers

(b) decrease the target correctness rate

(c) reduce the mean error rate

(d) consider strong-authentication

(e) opt for a different scheme

For item (a), we already have a very large number of lockers, increasing this number is not a favourable option. Target correctness should be as high as possible desirably, and hence item (b) is not an option either.

To reduce the mean error rate, item (c), we can work on the image preprocessing and feature extraction methods, e.g., opting for a different feature extraction algorithm or adjusting its parameters, and perhaps consider selecting a region of interest. Item (d) suggests authentication using at least two methods, e.g., a password or secure token. We discuss item (e) further in subsection 3.5.

3.4 Entropy estimation

Entropy allows us to estimate how much information we can use to distinguish between two different fingers.

f	TPR
3183467909	0.7000000000771347
4×10^9	0.7797036196692189
5×10^9	0.8490756890664359
6×10^9	0.8966022610241299
7×10^9	0.929162555991206
8×10^9	0.9514695048131558
9×10^9	0.9667519205974185
10×10^9	0.9772218523692289

Table 2: Example of parameters for an error rate of 0.418643 and k set to 40. The lowest f value for which $1 - \left(1 - (1 - 0.418643)^{40}\right)^f \geq 0.7$, is 3183467909

i	k	TPR
1	33	0.8145670783622886
2	34	0.859035228102637
3	35	0.8188683965816633
4	36	0.7340257036554612
5	36	0.8089928670684416
6	36	0.8628298849469516
7	37	0.7400759940878694
8	37	0.7855860591561721
9	37	0.8231277720314829

Table 3: For a fixed f taking values $i \times 10^8$, with $i = \{1 \ldots 9\}$, we find the maximum value of k for which $TPR > 0.7$

To estimate the entropy of the hashes, we take the approach defined in both [11, 17]. We compute the ratio of disagreeing bits, i.e., the Hamming distance, for each pair of hashes coming from different fingers this time. We used the hashes previously computed to estimate the mean error rate, yet instead of computing the Hamming distance between two hashes coming from the same finger, we compute the distance between two hashes coming from different fingers. For this data, we compute the mean and variance and obtain the following results $\mu = 0.464763, \sigma^2 = 0.000174$.

Assuming the obtained distribution fits a binomial distribution (Fig. 1 in Appendix) with the same mean and variance, then we can approximate the entropy of our data with the entropy of the binomial distribution using the following formulas:

$$N = \frac{\mu(1-\mu)}{\sigma^2}$$

$$H = -\mu \log_2 \mu - (1-\mu) \log_2 (1-\mu)$$

$$H_T = N \times H$$

We obtain $N = 1425.81, H_T = 1420.70$ and an entropy rate of 0.41.

3.5 New construction

We present a new construction with the hope of offering higher security and lower computation overhead than Construction 1.

For this, we propose a variant of a Digital Locker where the lock function is unlockable for two key values: the correct one v and a random one u. For the correct key v, it would unlock to the locked val, while for the other key u it unlocks to $val' \neq val$. That is $unlock(v, lock(v, val, u)) = val$ and $unlock(u, lock(v, val, u)) = val', val' \neq val$.

An example of such a locker is the following construction. To lock the random string r with the real key v and another (random) key u, we find a random h such that $H(h||v) \oplus H(h||u)$ ends with s zero bits. We then set $c = lock(v, r, u) = (h, H(h||v) \oplus (r||0^s))$. Let us denote $c = (c1, c2)$ for unlocking. To unlock c by using key v, we check that $c2 \oplus H(c1||v)$ ends with s zero bits. When this is the case, we set r to be the prefix.

The advantage of the modified Digital Locker over the original variant is that an adversary is able to unlock by performing an exhaustive search for the key but she does not know whether she found the real locked value or a random value, it can be either. We use this in the Fuzzy Extractor construction, and in contrast to Construction 1, we use secret sharing for the random string to be locked, r. We create f shares of the secret r, $r_1 \dots r_f$, and lock each r_i. The reconstruction of r succeeds if we can unlock at least t out of f r_i values. Please note that unlocking ($\leq$) $t-1$ shares of r leaves r completely undetermined. We present the new construction below, Construction 2.

Construction 2. (New construction) We use a secret sharing scheme with parameters t and f, and two algorithms *Share* and *Recover*. *Share* takes as input s and outputs $s_1, \dots, s_f$, while *Recover* takes as input a list of s_i. If the list contains at least t-out-of-f valid s_i values, it outputs s, otherwise, it outputs $\perp$.

Gen
Input: $w = w_1, \dots, w_n$
Sample $r \xleftarrow{\$} \{0,1\}^\ell$
Compute $Share(r) = r_1 \dots r_f$
For $i = 1 \dots f$:
  Choose random $1 \leq j_{i,1}, \dots, j_{i,k} \leq n$
  Set $v_i = w_{j_{i,1}}, \dots, w_{j_{i,k}}$
  Sample $u_i \xleftarrow{\$} \{0,1\}^k$
  Find h s.t. $H(h||v_i) \oplus H(h||u_i)$ ends with s zero bits
  Set $c_i = lock(v_i, r_i, u_i) = (h, H(h||v_i) \oplus (r_i||0^s))$
  Set $p_i = c_i, (j_{i,1}, \dots, j_{i,k})$
Output: (r, p), where $p = p_1 \dots p_f$

Rep
Input: $(w' = w'_1, \dots, w'_n, p = p_1 \dots p_f)$
Set $shares = list()$
For $i = 1, \dots, f$:
  Parse $p_i = c_i, (j_{i,1}, \dots, j_{i,k})$
  Set $v'_i = w'_{j_{i,1}}, \dots, w'_{j_{i,k}}$
  Set $z = unlock(v'_i, c_i) = H(c1_i||v'_i) \oplus c2_i$
  If z ends with 0^s:
    Set r_i to prefix of z
    Append r_i to $shares$
Output: $Recover(shares)$

We denote by er_s the per-bit Hamming distance when two readings come from the same finger, and by er_d when two readings come from different fingers. We suggest the following formulas:

1. 2^f should be large enough to avoid exhaustive search

2. $1 - \left(1 - 2^{-k}\right)^f$ should be small to avoid having a $k-$packet equal to the incorrect key by accident

3. $f \times (1 - er_s)^k > t$ to be able to extract with er_s

4. $f \times (1 - er_d)^k < t$ to be unable to extract with er_d

We leave the security proof for the new constructions, i.e., Reusable Fuzzy Extractor, respectively composable Digital Locker, for future work.

4 Alignment

Capturing biometric samples using a device without fixed support, e.g., using pegs, may result in differences between multiple readings. In our case, these arise from a different positioning of the finger. For example, a user may place her finger inside the device $2mm$ further as compared to the original reading, or $2°$ rotated to the left.

A property our system aims to satisfy is identifying a user displaying the same biometric feature despite the misalignment between the original reading/the model (stored as reference template either raw or in a processed form) and subsequent readings/a probe (used to identify the user).

While a Fuzzy Extractor is able to tolerate up to t errors between the model and a probe, we would like to allocate these t errors to actual variation of the features, e.g., finger-vein patterns, and not to the variation of the finger poses. To this end, we investigate which image processing actions we can take to reduce (ideally entire remove) the misalignment between two images. For the remainder of this section, let us denote the model by A and a probe by B.

We classify the proposed solutions according to the permission access they require and discuss each case individually:

1. access to the reference template

2. access to multiple instances of *Rep*

3. access to a single instance of *Rep*

4.1 Access to the reference template

Having access to the captured finger-vein image A provides the most opportunities for the transformations possible on finger-vein image B.

Literature shows that given a number, e.g., 3, 4, of control points, i.e., coordinates known precisely in both image A and image B, it is possible to compute the coefficients of the affine transformation matrix. Once the coefficients are known, they can be used to transform each pixel of image B, producing the desired image [18]. Determining these control points is another important task. One example in the literature for identification based on palm-dorsum uses the finger-webs as control points [19]. Another example for identification using finger-veins uses minutia points [20].

Determining the affine coefficients allows for the transformation to be applied to the entire image. This transformation accounts for changes in rotation, scaling and translation.

Hence, having access to finger-vein image A would allow us to follow a similar approach as in [20] and align finger-vein image B using the affine transformation.

The affine transformation transforms a point (x, y) to a point (x', y') via:

$$\begin{pmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{pmatrix} \times \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix}$$

To determine the coefficients of the affine transformation matrix, (a, b, c, d, e, f), three pairs of points are required.

$$\begin{pmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{pmatrix} \times \begin{pmatrix} x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \\ 1 & 1 & 1 \end{pmatrix} = \begin{pmatrix} x'_1 & x'_2 & x'_3 \\ y'_1 & y'_2 & y'_3 \\ 1 & 1 & 1 \end{pmatrix}$$

In [20] variation in the number and position of minutia points can exist (be it due to location, posture, lighting). To make sure one selects the same points in A and B, the authors consider all combinations from the detected bifurcation points, compute the coefficients for each such pair and when they belong to the predefined range of values, they will be accepted. Using the computed coefficients, the positions of other minutia points are modified. The affine matrix for which the average of minimum distances (average all minimum distances between minutia points in A and B) is minimal. This affine transformation matrix is used to transform the input image B.

Another example of a transformation is computing the Hamming distance between images A and B while processing image B (e.g., translation, rotation, scaling). Please note that image processing can also be performed over the frequency domain instead of the spatial domain (which yields better computational complexities). For instance, [21, 22] evaluate the translation parameters between two images using the Phase Only Correlation function. Phase Correlation is based on the translation property of FT, which states that a shift of two relevant images in the spatial domain is transformed into the frequency domain as phase differences [23].

4.2 Access to multiple instances of *Rep*

We now consider the case in which we do not have access directly to finger-vein image A, yet we have access to the Fuzzy Extractor primitive and an 'unlimited' number of calls to *Rep* procedure.

In this case, a naive solution is to attempt aligning finger-vein image B to finger-vein image A blindly, and use the feedback from the *Rep* procedure. In case the subsequent reading is within t errors from the reference reading, the correctness property of Fuzzy Extractor ensures that $\forall(w, w')$ s.t. $dis(w, w') \leq t$, $Gen(w)$=(R, P), prob. $Rep(w', P) = R$ at least $1 - \delta$ (if $dis > t$, no guarantees).

Consequently, we can perform transformations, e.g., translation, on image B and for each such transformation call the *Rep* procedure until *Rep* outputs a value.

The disadvantage of this solution is that it may require a large number of transformations to be performed. Moreover, a predefined setting for the type and number of transformations should be set so that the algorithm does not run forever. It can be the case that image B is actually of a different user as compared to image A and hence, regardless of the transformation performed the *Rep* procedure should not yield a result. From here, we also point out that transformations should make sense, e.g., permuting all bits of an image up until the *Rep* procedure outputs a result does not make sense. Last but not least, having access to 'unlimited' calls to *Rep* might be unrealistic. Consider an application where the output of the *Rep* procedure is directly used to log in to a website. In this case, the log in may have a limited number of attempts, e.g., 6.

4.3 Access to a single instance of *Rep*

A more elegant approach is to minimise the number of calls to *Rep* (ideally a single call). To this end, we describe (and implement) three methods.

First, we use the centre of mass to align the images. After the feature extraction is performed, one obtains a matrix with 0 and 1 values. The centre of mass is computed as the average of white points. It iterates through all values of the image matrix, whenever it encounters value 1, it adds the position of the corresponding coordinate (x, y) to the sum (component-wise) and at the end divide this sum by the total number of white pixels. The output coordinate represents the centre of mass. For each input image (extracted features) we compute this coordinate and align the image such that the centre of mass becomes the new physical centre of the image[1].

Second, we follow the approach of [11, 24] which first localises the finger region and then aligns the finger to the centre of the image. The estimation of the finger region is performed using a mask filter technique, which returns the binary mask of the finger region. Using the image and its mask, we apply a normalisation that aligns the finger to the centre of the image using a more limited affine transformation. One computes the centre for each edge (column-wise) and calculates the best linear fit parameters for a straight line passing through those points. The line parameters are used to define the rotation angle and translation on the $y-$axis.

Third, we present an additional alignment technique based on the second method. The method is the same as previously described yet we modify the normalisation procedure. In addition, we perform a horizontal translation using an independent method, identifying the fingertip location and shifting it to a predefined coordinate.

Finally, we define a function that shifts each image according to a predefined

[1] Please note that the physical centre is constant, i.e., the same for each image

point. We set a predefined point by experimental observation. We notice that the top left-most corner of the mask is close to $(45, 75)$. Let this be the point to which we shift our image to, $P = (45, 75)$. We compute the difference between the first white pixel in the top left corner of the mask say M and our point P. We use the resulting (x, y) value as coefficients for affine transform with which we translate both the image and the mask.

4.4 Evaluation of proposed methods

We present an evaluation of the four alignment methods. We define the method set $\{method_1, method_2, method_3, method_4\}$ where index $method_i$ refers to the methods described in the previous subsection (except $method_4$ which we redefine below). Moreover, we use *none* to indicate no alignment function is used and *optimal* to indicate the alignment given by *miurascore* function. This function outputs the best similarity score between the model and a probe by shifting the probe in (x, y) direction.

By experimental analysis, we observe $method_4$ is not sufficiently robust. Aligning by the top-left point of the masks fails when a mask has an irregular shape. Since the value of this corner can vary too much, we modify the function as follows. We localise position i on the x-axis as the first edge of the mask with sufficiently many points and shift on x-axis by $ct_x - i$ where ct_x is a constant value. This way, an outlier point does not (wrongly) influence the resulting image.

We consider intraclass comparisons. For each method $method_i$, we compute the Hamming distance between data from the same finger. This is similar to the computation in subsection 3.3, yet this time on the finger-vein pattern image instead of the biometric hash of the finger-vein pattern image. We consider the model and a probe for each person, i.e., same finger, same hand, same camera. We use the finger-vein data obtained after the pre-processing and the feature extraction process. The finger-vein pattern is represented as a $2D$ matrix of shape $(240, 376)$, yielding $\{0, 1\}^{90240}$.

To compute the Hamming distance between two patterns, we compute the XOR between the model and probe matrices and output the number of 1s out of 90240. Whenever the values are the same in the model and the probe, the XOR between them is 0. Hence, the number of 1s gives the difference between the two. We report the Hamming distance values in Table 4.

To decide which of the four methods performs best, we can make use of the *optimal* and *none* columns as reference points to compare against. For instance, in Table 5 we compute the percentage change of each method:

$$\frac{HD_{method_i} - HD_{reference}}{HD_{reference}} \times 100$$

where $reference$ takes values *none* and *optimal*.

A positive percentage indicates an increase in Hamming distance over $reference$ when performing alignment using $method_i$. When negative, it denotes a decrease in Hamming distance. We seek the method with the highest decrease,

pair	*none*	$method_1$	$method_2$	$method_3$	$method_4$	*optimal*
0	0.070191	0.073493	0.072404	0.072404	**0.06988**	0.063298
1	0.070623	0.071288	**0.06715**	**0.06715**	0.071243	0.062932
2	0.06516	0.071875	**0.067223**	0.067969	0.069127	0.063608
3	0.059453	0.069515	0.064984	0.064984	**0.059652**	0.058943
4	0.069902	0.072496	**0.069326**	**0.069326**	0.070855	0.070058
5	0.063032	0.068728	**0.065389**	**0.065389**	0.067154	0.063254
6	0.069714	**0.067609**	0.067617	0.068301	0.069714	0.066013
7	0.061126	0.066622	**0.061565**	**0.061565**	0.064162	0.061126
8	0.082491	0.083134	**0.081067**	0.08144	0.081715	0.079743
9	0.087212	0.097762	0.089482	**0.087326**	0.089849	0.075598
10	0.082214	0.084541	**0.079202**	**0.079202**	0.080929	0.078092
11	0.077183	0.084342	0.077648	0.077648	**0.077183**	0.066102
12	0.083566	0.083078	**0.079513**	**0.079513**	0.08463	0.073681
13	0.086469	0.087844	**0.08713**	**0.08713**	0.088952	0.081704
14	0.071609	0.072451	0.072539	0.072539	**0.071609**	0.061325
15	0.077405	0.078579	**0.074653**	**0.074653**	0.075609	0.068739
16	0.057502	0.063952	**0.054995**	0.05514	0.057746	0.053557
17	0.097418	0.09693	**0.094995**	0.095057	0.097662	0.093562
18	0.084752	0.084087	**0.082332**	**0.082332**	0.084752	0.070944
19	0.084752	0.08719	**0.08344**	**0.08344**	0.084198	0.08156

Table 4: The table shows the Hamming distance between the model and probe. *none* indicates no image transformation for neither the model nor the probe, $method_1$ shifts the physical centre of each image to the centre of mass, $method_2$ translates on y-axis and rotates according to the line parameters that pass through the column-wise centre of the finger contour, $method_3$ in addition to $method_2$, it translates on x-axis using fingertip position, $method_4$ shifts each image on the x-axis using the leftmost edge, *optimal* shifts only the probe by the coordinates (x, y) obtained from *miurascore*. In bold we highlight the best method, i.e., $\min_i(method_i - optimal) = \min_i(method_i - none) = \min_i(method_i)$.

respectively the least increase in Hamming distance when the sign is positive. When comparing to *none*, we wish the Hamming distance to be lower than when no method applied. Of course, this is also desired with respect to the *optimal*, yet we see this is not the case so we settle for the $method_i$ with the least increase in Hamming distance. As a general comment, we can always rely on the Hamming distance being closest to 0 as this is the case when the image perfectly match. We notice that $method_2$ performs best in 14 out of 20 cases while $method_1$ perform worst.

We would also like to emphasise that even though we denoted *optimal* alignment, this method is not ideal. The Hamming distance may be close to 0, yet this is because the finger-vein pattern contains many elements. For instance, the model has 3526 1s, the probe has 3766 1s while their XOR has 5712 1s. This shows that the finger-vein patterns actually do not overlap but rather accumulate (see Table 6). It can (partly) be due to an actual variation in the finger-vein patterns, yet it can also be due to other image transformations such as rotation or scaling.

$method_1$	$method_2$	$method_3$	$method_4$	$method_1$	$method_2$	$method_3$	$method_4$
4.70	3.15	3.15	-0.44	16.11	14.39	14.39	10.40
0.94	-4.92	-4.92	0.88	13.28	6.70	6.70	13.21
10.31	3.17	4.31	6.09	13.00	5.68	6.86	8.68
16.92	9.30	9.30	0.33	17.94	10.25	10.25	1.20
3.71	-0.82	-0.82	1.36	3.48	-1.04	-1.04	1.14
9.04	3.74	3.74	6.54	8.65	3.38	3.38	6.17
-3.02	-3.01	-2.03	0.00	2.42	2.43	3.47	5.61
8.99	0.72	0.72	4.97	8.99	0.72	0.72	4.97
0.78	-1.73	-1.27	-0.94	4.25	1.66	2.13	2.47
12.10	2.60	0.13	3.02	29.32	18.37	15.51	18.85
2.83	-3.66	-3.66	-1.56	8.26	1.42	1.42	3.63
9.28	0.60	0.60	0.00	27.59	17.47	17.47	16.76
-0.58	-4.85	-4.85	1.27	12.75	7.92	7.92	14.86
1.59	0.76	0.76	2.87	7.51	6.64	6.64	8.87
1.18	1.30	1.30	0.00	18.14	18.29	18.29	16.77
1.52	-3.56	-3.56	-2.32	14.32	8.60	8.60	9.99
11.22	-4.36	-4.11	0.42	19.41	2.68	2.96	7.82
-0.50	-2.49	-2.42	0.25	3.60	1.53	1.60	4.38
-0.78	-2.86	-2.86	0.00	18.53	16.05	16.05	19.46
2.88	-1.55	-1.55	-0.65	6.90	2.31	2.31	3.23

Table 5: The table indicates the percentage change in relation to no alignment method applied (left-hand side) and to the optimal shifting parameters (right-hand side). A positive percentage denotes an increase in Hamming distance when $method_i$ is applied as compared to the reference; while a negative percentage denotes a decrease in Hamming distance. The highest decrease, respectively the least increase when the sign is positive, indicates the best strategy.

pair	n_A	n_B	$n_{A \oplus B}$	pair	n_A	n_B	$n_{A \oplus B}$
0	3526	3766	5712	10	4212	4397	7047
1	3524	3573	5679	11	4389	4836	5965
2	3222	3544	5740	12	4229	4568	6649
3	3608	3273	5319	13	4531	5146	7373
4	3508	3748	6322	14	3841	3791	5534
5	3808	3298	5708	15	4279	4642	6203
6	3552	3889	5957	16	3752	3333	4833
7	3504	3536	5516	17	5084	5405	8443
8	3971	4691	7196	18	4031	4377	6402
9	4963	5417	6822	19	4596	5018	7360

Table 6: The number of 1s in i: n_i, where A the finger-vein pattern of the model, B the shifted finger-vein pattern of the probe. The probe has been shifted by the translation coordinates (x, y) given by the *miurascore* function. The function indicates the shift for which the probe matches best the model. When A and B match, the number of 1s in their XOR should be 0.

5 Conclusion

We approach two challenges on the topic of biometric authentication by using finger-veins. The first is selecting a Fuzzy Extractor construction that suits our needs: can be applied to human biometrics, is reusable, is hard to invert. We present the chosen Reusable Fuzzy Extractor construction and opt for the biometric hash of finger-vein patterns as the input source. This is because the construction assumes the source has high-entropy samples. We estimate parameters for this scheme and obtain a sample size of $k = 40$ and $f = 3.2 \times 10^9$ Digital Lockers. This means the $(40-\text{bit})$ security is too low and the number of lockers is too high. We propose ways to improve on these results, i.e., increase the security and lower time and storage.

We also present ways in which the noise among different readings of the same finger can be reduced. We propose four methods to reduce the misalignment between the model and a probe without having access to the model and show which of these is preferable. The method that performs best is $method_2$. Given a finger-vein pattern image, $method_2$ translates it on the $y-$axis and rotates it according to the line parameters that pass through the column-wise centre of the finger contour.

5.1 Future work

There are multiple directions the project can continue. One of the most important is having access to biometric data and estimating the entropy for the specific dataset and (possibly different) feature extraction method(s). A few articles that can help with this direction are [17, 11]

Of course, a richer performance evaluation should be performed once there is access to data. In case the estimation of parameters for Construction 1 still yields a low security and requires high computation, one can further investigate the proposals at the end of subsection 3.3. Another evaluation can be performed for the proposed approaches in dealing with the assumptions on the source, subsection 3.1.

Finally, having access to both the model and probe would significantly increase the possibilities for image transformations, e.g. [20], which should perhaps be considered. This can consequently have a positive impact on the mean error rate and the parameters of Fuzzy Extractor constructions.

References

[1] Ran Canetti, Benjamin Fuller, Omer Paneth, Leonid Reyzin, and Adam Smith. Reusable fuzzy extractors for low-entropy distributions. Cryptology ePrint Archive, Report 2014/243, 2014. `https://ia.cr/2014/243`.

[2] Yevgeniy Dodis, Rafail Ostrovsky, Leonid Reyzin, and Adam Smith. Fuzzy extractors: How to generate strong keys from biometrics and other noisy

data. Cryptology ePrint Archive, Report 2003/235, 2003. https://ia.cr/2003/235.

[3] Benjamin Fuller, Xianrui Meng, and Leonid Reyzin. Computational fuzzy extractors. Cryptology ePrint Archive, Report 2013/416, 2013. https://ia.cr/2013/416.

[4] Nir Bitansky and Ran Canetti. On strong simulation and composable point obfuscation. Cryptology ePrint Archive, Report 2010/414, 2010. https://ia.cr/2010/414.

[5] Quentin Alamélou, Paul-Edmond Berthier, Chloé Cachet, Stéphane Cauchie, Benjamin Fuller, Philippe Gaborit, and Sailesh Simhadri. Pseudoentropic isometries: A new framework for fuzzy extractor reusability. Cryptology ePrint Archive, Report 2016/1100, 2016. https://ia.cr/2016/1100.

[6] Benjamin Fuller, Leonid Reyzin, and Adam Smith. When are fuzzy extractors possible? Cryptology ePrint Archive, Report 2014/961, 2014. https://ia.cr/2014/961.

[7] Yunhua Wen, Shengli Liu, and Shuai Han. Reusable fuzzy extractor from the decisional diffie—hellman assumption. *Des. Codes Cryptography*, 86(11):2495–2512, nov 2018.

[8] Yunhua Wen and Shengli Liu. Robustly reusable fuzzy extractor from standard assumptions. Cryptology ePrint Archive, Report 2018/818, 2018. https://ia.cr/2018/818.

[9] Yunhua Wen, Shengli Liu, and Dawu Gu. Generic constructions of robustly reusable fuzzy extractor. Cryptology ePrint Archive, Report 2019/018, 2019. https://ia.cr/2019/018.

[10] Pim Tuyls and Jasper Goseling. Capacity and examples of template-protecting biometric authentication systems. In Davide Maltoni and Anil K. Jain, editors, *Biometric Authentication*, pages 158–170, Berlin, Heidelberg, 2004. Springer Berlin Heidelberg.

[11] Vedrana Krivokuća and Sébastien Marcel. Towards quantifying the entropy of fingervein patterns across different feature extractors. In *2018 IEEE 4th International Conference on Identity, Security, and Behavior Analysis (ISBA)*, Jan 2018.

[12] Kenji Yasunaga and Kosuke Yuzawa. On the limitations of computational fuzzy extractors. Cryptology ePrint Archive, Report 2014/605, 2014. https://ia.cr/2014/605.

[13] John Daugman. How iris recognition works, 2004. https://www.cl.cam.ac.uk/~jgd1000/irisrecog.pdf.

[14] Xavier Boyen. Reusable cryptographic fuzzy extractors. Cryptology ePrint Archive, Report 2004/358, 2004. https://ia.cr/2004/358.

[15] Charles Herder, Ling Ren, Marten van Dijk, Meng-Day (Mandel) Yu, and Srinivas Devadas. Trapdoor computational fuzzy extractors and stateless cryptographically-secure physical unclonable functions. Cryptology ePrint Archive, Report 2014/938, 2014. https://ia.cr/2014/938.

[16] Daniel Apon, Chongwon Cho, Karim Eldefrawy, and Jonathan Katz. Efficient, reusable fuzzy extractors from lwe. Cryptology ePrint Archive, Report 2017/755, 2017. https://ia.cr/2017/755.

[17] Benjamin Fuller, Sailesh Simhadri, and James Steel. Reusable authentication from the iris. Cryptology ePrint Archive, Report 2017/1177, 2017. https://ia.cr/2017/1177.

[18] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing (3rd Edition)*. Prentice-Hall, Inc., USA, 2006.

[19] C.-L Lin, H.-F Chiang, H.-Y Cheng, C.-W Kuo, and J.-W Hsieh. An approach to aligning regions of interest on infrared palm-dorsum images captured in contactless scenarios. *Journal of Marine Science and Technology (Taiwan)*, 26:73–83, 01 2018.

[20] Eui Chul Lee, Hyeonchang Lee, and Kang Park. Finger vein recognition using minutia-based alignment and local binary pattern-based feature extraction. *International Journal of Imaging Systems and Technology*, 19:179 – 186, 09 2009.

[21] Koichi Ito, Student Member, Hiroshi Nakajima, Koji Nonmember, Takafumi Kobayashi, Tatsuo Aoki, and Higuchi. A fingerprint matching algorithm using phase-only correlation. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, E87-A, 03 2004.

[22] Mohd Shahrimie Mohd Asaari, Shahrel A. Suandi, and Bakhtiar Affendi Rosdi. Fusion of band limited phase only correlation and width centroid contour distance for finger based biometrics. *Expert Systems with Applications*, 41(7):3367–3382, 2014.

[23] Xiaohua Tong, Zhen Ye, Yusheng Xu, Sa Gao, Huan Xie, Qian Du, Shijie Liu, Xiong Xu, Sicong Liu, Kuifeng Luan, and Uwe Stilla. Image registration with fourier-based image correlation: A comprehensive review of developments and applications. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 12(10):4062–4081, 2019.

[24] Beining Huang, Yanggang Dai, Rongfeng Li, Darun Tang, and Wenxin Li. Finger-vein authentication based on wide line detector and pattern normalization. In *2010 20th International Conference on Pattern Recognition*, pages 1269–1272, 2010.

[25] A. Anjos, M. Günther, T. de Freitas Pereira, P. Korshunov, A. Mohammadi, and S. Marcel. Continuously reproducing toolchains in pattern recognition and machine learning experiments. In *International Conference on Machine Learning (ICML)*, August 2017.

Appendix

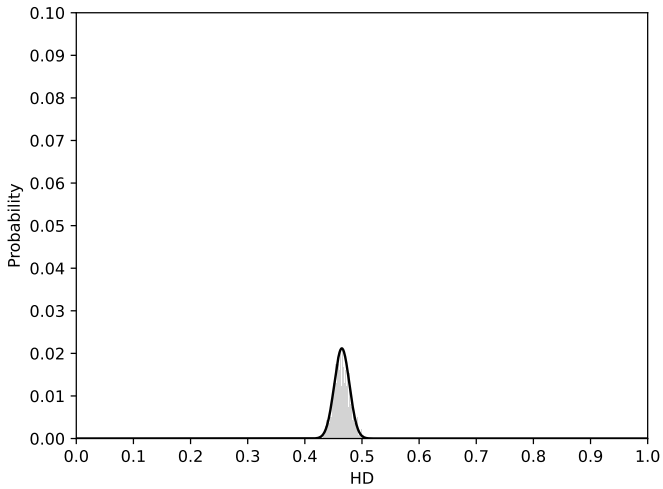

Figure 1: Hamming distance distribution and the corresponding binomial distribution overlaid. Plot generated with [25].

We explicitly state the data represented by the pair identifier $i = 0 \ldots 19$:

- (0, '5_right_middle_1_cam1.png', '5_right_middle_2_cam1.png')
- (1, '5_right_middle_1_cam2.png', '5_right_middle_2_cam2.png')
- (2, '5_left_ring_1_cam1.png', '5_left_ring_2_cam1.png')
- (3, '5_right_ring_1_cam1.png', '5_right_ring_2_cam1.png')
- (4, '5_left_ring_1_cam2.png', '5_left_ring_2_cam2.png')
- (5, '5_right_ring_1_cam2.png', '5_right_ring_2_cam2.png')
- (6, '5_right_index_1_cam1.png', '5_right_index_2_cam1.png')
- (7, '5_right_index_1_cam2.png', '5_right_index_2_cam2.png')
- (8, '3_left_middle_1_cam1.png', '3_left_middle_2_cam1.png')

- (9, ‘3_right_middle_1_cam1.png’, ‘3_right_middle_2_cam1.png’)
- (10, ‘3_left_middle_1_cam2.png’, ‘3_left_middle_2_cam2.png’)
- (11, ‘3_right_middle_1_cam2.png’, ‘3_right_middle_2_cam2.png’)
- (12, ‘3_left_ring_1_cam1.png’, ‘3_left_ring_2_cam1.png’)
- (13, ‘3_right_ring_1_cam1.png’, ‘3_right_ring_2_cam1.png’)
- (14, ‘3_left_ring_1_cam2.png’, ‘3_left_ring_2_cam2.png’)
- (15, ‘3_right_ring_1_cam2.png’, ‘3_right_ring_2_cam2.png’)
- (16, ‘3_left_index_1_cam1.png’, ‘3_left_index_2_cam1.png’)
- (17, ‘3_right_index_1_cam1.png’, ‘3_right_index_2_cam1.png’)
- (18, ‘3_left_index_1_cam2.png’, ‘3_left_index_2_cam2.png’)
- (19, ‘3_right_index_1_cam2.png’, ‘3_right_index_2_cam2.png’)